Unit 5

Hyperledger Blockchain

Introduction

- In general terms, a blockchain is an immutable transaction ledger, maintained within a distributed network of peer nodes.
- These nodes each maintain a copy of the ledger by applying transactions that have been validated by a consensus protocol, grouped into blocks that include a hash that bind each block to the preceding block.

Introduction

- The first and most widely recognized application of blockchain is the Bitcoin cryptocurrency, though others have followed in its footsteps.
- Ethereum, an alternative cryptocurrency, took a different approach, integrating many of the same characteristics as Bitcoin but adding smart contracts to create a platform for distributed applications.
- Bitcoin and Ethereum fall into a class of blockchain that we would classify as public permissionless blockchain technology.
- Basically, these are public networks, open to anyone, where participants interact anonymously.

Introduction

- As the popularity of Bitcoin, Ethereum and a few other derivative technologies grew, interest in applying the underlying technology of the blockchain, distributed ledger and distributed application platform to more innovative enterprise use cases also grew.
- However, many enterprise use cases require performance characteristics that the permissionless blockchain technologies are unable (presently) to deliver.
- In addition, in many use cases, the identity of the participants is a hard requirement, such as in the case of financial transactions where Know-Your-Customer (KYC) and Anti-Money Laundering (AML) regulations must be followed.

Introduction

- For enterprise use, we need to consider the following requirements:
  - Participants must be identified/identifiable
  - Networks need to be *permissioned*
  - High transaction throughput performance
  - Low latency of transaction confirmation
  - Privacy and confidentiality of transactions and data pertaining to business transactions

Hyperledger Fabric

- Hyperledger Fabric is an open source enterprise-grade permissioned distributed ledger technology (DLT) platform, designed for use in enterprise contexts, that delivers some key differentiating capabilities over other popular distributed ledger or blockchain platforms.
- One key point of differentiation is that Hyperledger was established under the Linux Foundation, which itself has a long and very successful history of nurturing open source projects under **open governance** that grow strong sustaining communities and thriving ecosystems. Hyperledger is governed by a diverse technical steering committee, and the Hyperledger Fabric project by a diverse set of maintainers from multiple organizations. It has a development community that has grown to over 35 organizations and nearly 200 developers since its earliest commits.

Modular and Configurable

- Fabric has a highly **modular** and **configurable** architecture, enabling innovation, versatility and optimization for a broad range of industry use cases including banking, finance, insurance, healthcare, human resources, supply chain and even digital music delivery.

- Fabric is the first distributed ledger platform to support smart contracts authored in general-purpose programming languages such as Java, Go and Node.js, rather than constrained domain-specific languages (DSL). This means that most enterprises already have the skill set needed to develop smart contracts, and no additional training to learn a new language or DSL is needed.

Permissioned

- The Fabric platform is also **permissioned**, meaning that, unlike with a public permissionless network, the participants are known to each other, rather than anonymous and therefore fully untrusted.
- This means that while the participants may not *fully* trust one another (they may, for example, be competitors in the same industry), a network can be operated under a governance model that is built off of what trust *does* exist between participants, such as a legal agreement or framework for handling disputes.

Pluggable consensus protocols

- One of the most important of the platform's differentiators is its support for **pluggable consensus protocols** that enable the platform to be more effectively customized to fit particular use cases and trust models.
- For instance, when deployed within a single enterprise, or operated by a trusted authority, fully byzantine fault tolerant consensus might be considered unnecessary and an excessive drag on performance and throughput.
- In situations such as that, a crash fault-tolerant (CFT) consensus protocol might be more than adequate whereas, in a multi-party, decentralized use case, a more traditional byzantine fault tolerant (BFT) consensus protocol might be required.

- Fabric can leverage consensus protocols that **do not require a native cryptocurrency** to incent costly mining or to fuel smart contract execution.
- Avoidance of a cryptocurrency reduces some significant risk/attack vectors, and absence of cryptographic mining operations means that the platform can be deployed with roughly the same operational cost as any other distributed system.

Privacy and confidentiality

- The combination of these differentiating design features makes Fabric one of the **better performing platforms** available today both in terms of transaction processing and transaction confirmation latency, and it enables **privacy and confidentiality** of transactions and the smart contracts (what Fabric calls “chaincode”) that implement them.

Modularity

- Hyperledger Fabric has been specifically architected to have a modular architecture. Whether it is pluggable consensus, pluggable identity management protocols such as LDAP or OpenID Connect, key management protocols or cryptographic libraries, the platform has been designed at its core to be configured to meet the diversity of enterprise use case requirements.
  - At a high level, Fabric is comprised of the following modular components:
  - A pluggable ordering service establishes consensus on the order of transactions and then broadcasts blocks to peers.
  - A pluggable membership service provider is responsible for associating entities in the network with cryptographic identities.
  - An optional peer-to-peer gossip service disseminates the blocks output by ordering service to other peers.
  - Smart contracts (“chaincode”) run within a container environment (e.g. Docker) for isolation. They can be written in standard programming languages but do not have direct access to the ledger state.
  - The ledger can be configured to support a variety of DBMSs.
  - A pluggable endorsement and validation policy enforcement that can be independently configured per application.

Permissioned vs Permissionless Blockchains

- In a permissionless blockchain, virtually anyone can participate, and every participant is anonymous. In such a context, there can be no trust other than that the state of the blockchain, prior to a certain depth, is immutable. In order to mitigate this absence of trust, permissionless blockchains typically employ a “mined” native cryptocurrency or transaction fees to provide economic incentive to offset the extraordinary costs of participating in a form of byzantine fault tolerant consensus based on “proof of work” (PoW).

- Permissioned blockchains, on the other hand, operate a blockchain amongst a set of known, identified and often vetted participants operating under a governance model that yields a certain degree of trust. A permissioned blockchain provides a way to secure the interactions among a group of entities that have a common goal but which may not fully trust each other. By relying on the identities of the participants, a permissioned blockchain can use more traditional crash fault tolerant (CFT) or byzantine fault tolerant (BFT) consensus protocols that do not require costly mining.

Permissioned vs Permissionless Blockchains

- Additionally, in such a permissioned context, the risk of a participant intentionally introducing malicious code through a smart contract is diminished. First, the participants are known to one another and all actions, whether submitting application transactions, modifying the configuration of the network or deploying a smart contract are recorded on the blockchain following an endorsement policy that was established for the network and relevant transaction type. Rather than being completely anonymous, the guilty party can be easily identified and the incident handled in accordance with the terms of the governance model.

Smart Contracts

- A smart contract, or what Fabric calls “chaincode”, functions as a trusted distributed application that gains its security/trust from the blockchain and the underlying consensus among the peers. It is the business logic of a blockchain application.
- There are three key points that apply to smart contracts, especially when applied to a platform:
- many smart contracts run concurrently in the network,
- they may be deployed dynamically (in many cases by anyone), and
- application code should be treated as untrusted, potentially even malicious.
- Most existing smart-contract capable blockchain platforms follow an **order-execute** architecture in which the consensus protocol:
- validates and orders transactions then propagates them to all peer nodes,
- each peer then executes the transactions sequentially.
- The order-execute architecture can be found in virtually all existing blockchain systems, ranging from public/permissionless platforms such as Ethereum (with PoW-based consensus) to permissioned platforms such as Tendermint, Chain, and Quorum.

Smart Contracts

- Smart contracts executing in a blockchain that operates with the order-execute architecture must be deterministic; otherwise, consensus might never be reached.
- To address the non-determinism issue, many platforms require that the smart contracts be written in a non-standard, or domain-specific language (such as Solidity) so that non-deterministic operations can be eliminated.
- This hinders wide-spread adoption because it requires developers writing smart contracts to learn a new language and may lead to programming errors.
- Further, since all transactions are executed sequentially by all nodes, performance and scale is limited.
- The fact that the smart contract code executes on every node in the system demands that complex measures be taken to protect the overall system from potentially malicious contracts in order to ensure resiliency of the overall system.

A New Approach

- Fabric introduces a new architecture for transactions that we call **execute-order-validate**. It addresses the resiliency, flexibility, scalability, performance and confidentiality challenges faced by the order-execute model by separating the transaction flow into three steps:
  - *execute* a transaction and check its correctness, thereby endorsing it,
  - *order* transactions via a (pluggable) consensus protocol, and
  - *validate* transactions against an application-specific endorsement policy before committing them to the ledger
- This design departs radically from the order-execute paradigm in that Fabric executes transactions before reaching final agreement on their order.

A New Approach

- In Fabric, an application-specific endorsement policy specifies which peer nodes, or how many of them, need to vouch for the correct execution of a given smart contract. Thus, each transaction need only be executed (endorsed) by the subset of the peer nodes necessary to satisfy the transaction's endorsement policy.
- This allows for parallel execution increasing overall performance and scale of the system. This first phase also **eliminates any non-determinism**, as inconsistent results can be filtered out before ordering.
- Because we have eliminated non-determinism, Fabric is the first blockchain technology that **enables use of standard programming languages**.

Privacy and Confidentiality

- As we have discussed, in a public, permissionless blockchain network that leverages PoW for its consensus model, transactions are executed on every node.
- This means that neither can there be confidentiality of the contracts themselves, nor of the transaction data that they process.
- Every transaction, and the code that implements it, is visible to every node in the network. In this case, we have traded confidentiality of contract and data for byzantine fault tolerant consensus delivered by PoW.
- This lack of confidentiality can be problematic for many business/enterprise use cases.
- For example, in a network of supply-chain partners, some consumers might be given preferred rates as a means of either solidifying a relationship, or promoting additional sales.
- If every participant can see every contract and transaction, it becomes impossible to maintain such business relationships in a completely transparent network — everyone will want the preferred rates!

Privacy and Confidentiality

- As a second example, consider the securities industry, where a trader building a position (or disposing of one) would not want her competitors to know of this, or else they will seek to get in on the game, weakening the trader's gambit.

- In order to address the lack of privacy and confidentiality for purposes of delivering on enterprise use case requirements, blockchain platforms have adopted a variety of approaches. All have their trade-offs.

- Encrypting data is one approach to providing confidentiality; however, in a permissionless network leveraging PoW for its consensus, the encrypted data is sitting on every node. Given enough time and computational resource, the encryption could be broken. For many enterprise use cases, the risk that their information could become compromised is unacceptable.

Privacy and Confidentiality

- Zero knowledge proofs (ZKP) are another area of research being explored to address this problem, the trade-off here being that, presently, computing a ZKP requires considerable time and computational resources. Hence, the trade-off in this case is performance for confidentiality.

- In a permissioned context that can leverage alternate forms of consensus, one might explore approaches that restrict the distribution of confidential information exclusively to authorized nodes.

- Hyperledger Fabric, being a permissioned platform, enables confidentiality through its channel architecture and private data feature. In channels, participants on a Fabric network establish a sub-network where every member has visibility to a particular set of transactions. Thus, only those nodes that participate in a channel have access to the smart contract (chaincode) and data transacted, preserving the privacy and confidentiality of both. Private data allows collections between members on a channel, allowing much of the same protection as channels without the maintenance overhead of creating and maintaining a separate channel.

Pluggable Consensus

- The ordering of transactions is delegated to a modular component for consensus that is logically decoupled from the peers that execute transactions and maintain the ledger. Specifically, the ordering service. Since consensus is modular, its implementation can be tailored to the trust assumption of a particular deployment or solution. This modular architecture allows the platform to rely on well-established toolkits for CFT (crash fault-tolerant) or BFT (byzantine fault-tolerant) ordering.

- Fabric currently offers a CFT ordering service implementation based on the etcd library of the Raft protocol. Note also that these are not mutually exclusive. A Fabric network can have multiple ordering services supporting different applications or application requirements.

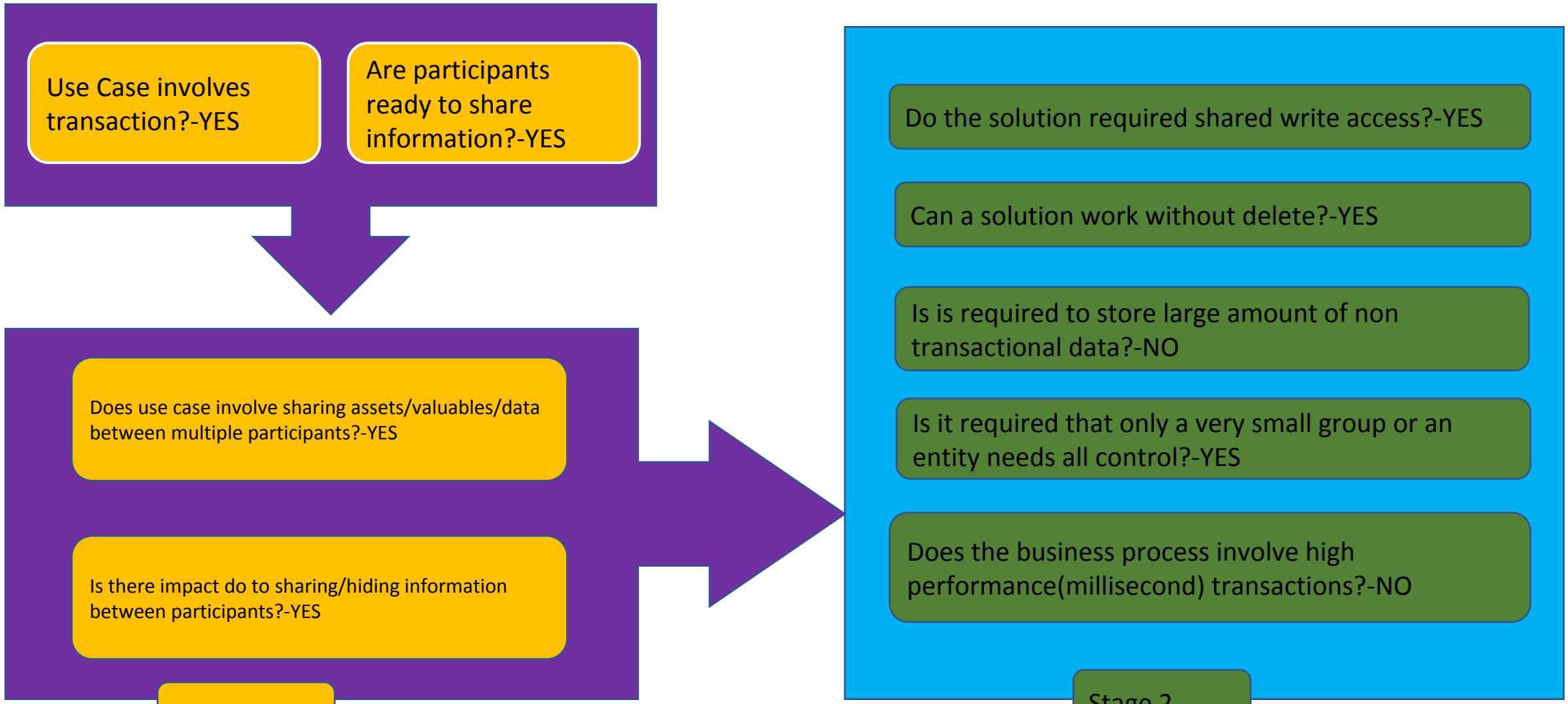
Performance and Scalability

- Performance of a blockchain platform can be affected by many variables such as transaction size, block size, network size, as well as limits of the hardware, etc.
- The Hyperledger Fabric Performance and Scale working group currently works on a benchmarking framework called Hyperledger Caliper.

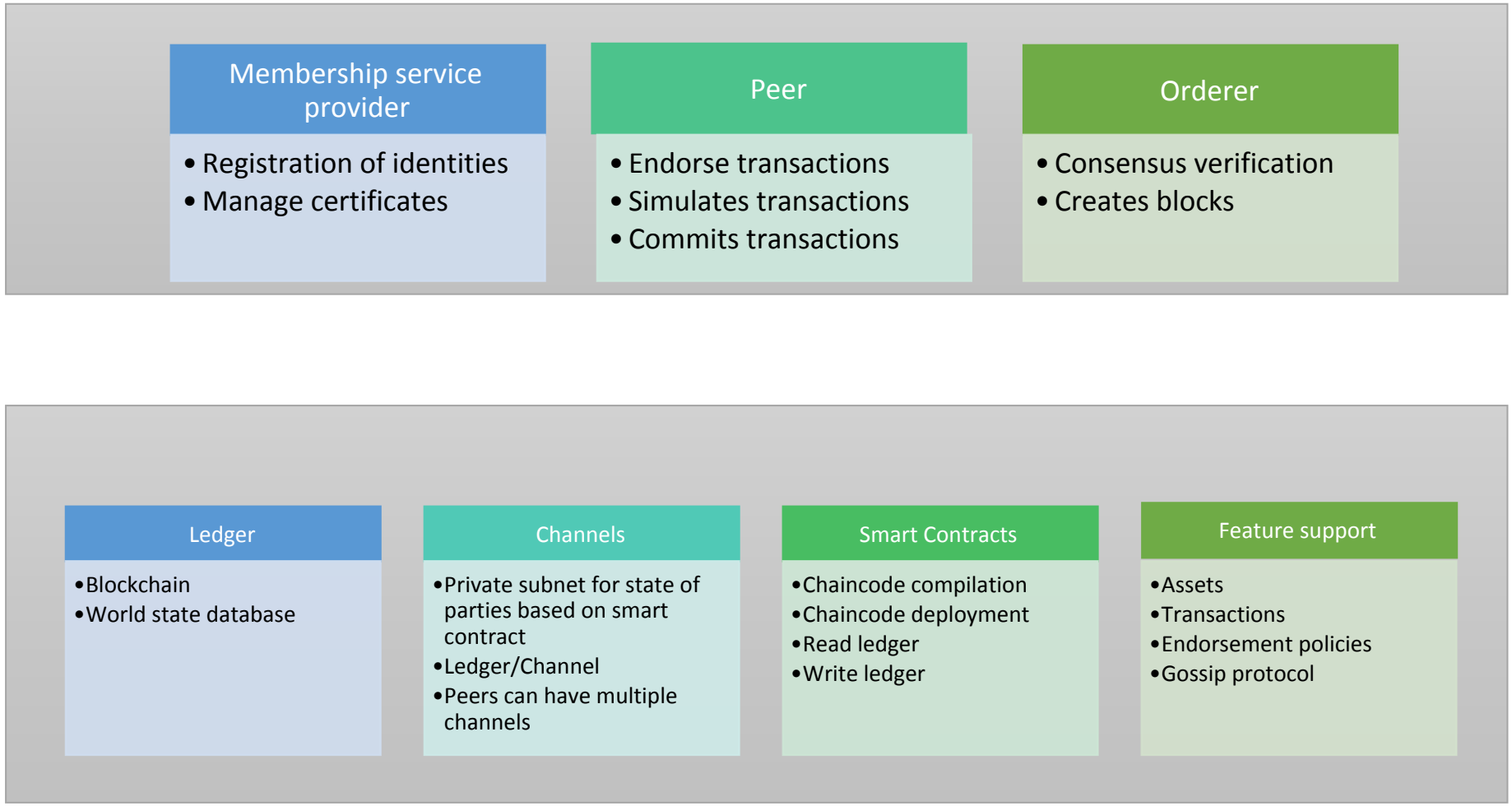
Car Ownership Tracking Use case

- Car manufacturing to customer use case
- Players may be car manufacturer, distributor,dealer, agent,customer, transport authority, insurer etc.
- For the sake of simplicity, we will consider only three manufacturer, dealer, customer.

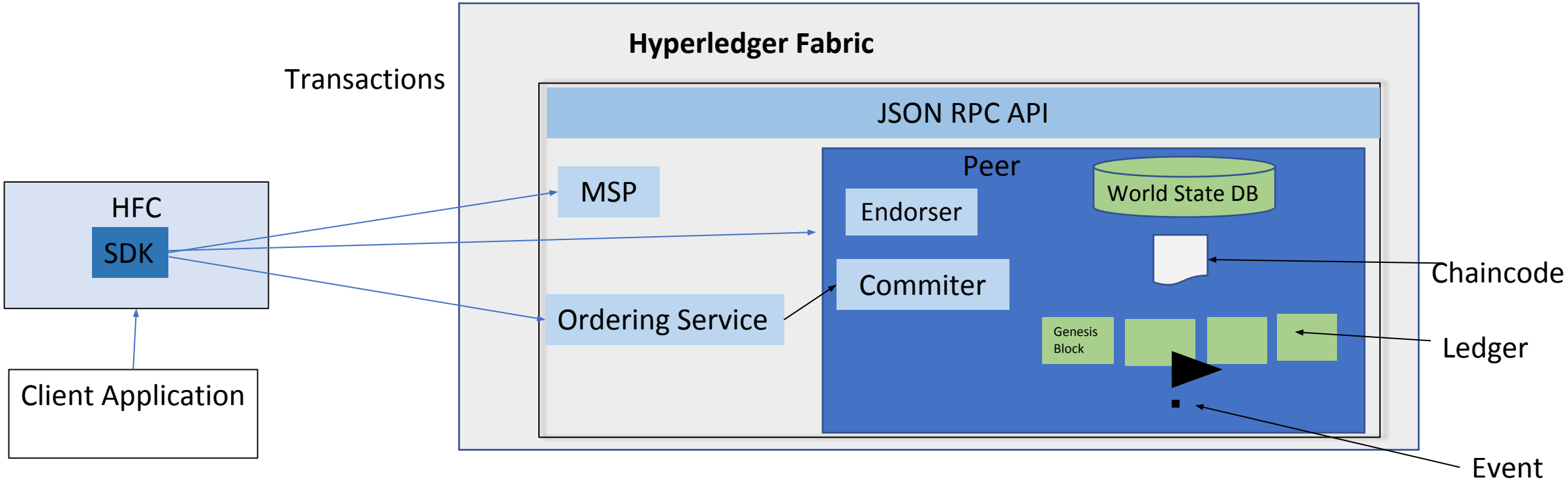
Blockchain Relevance Evaluation Framework



Hyperledger fabric components



Hyperledger Solution Components



Hyperledger Solution Components

- A membership service provider issues identity to each component within the network.
- A peer node can act as a endorser node to endorse transactions or a committer node to commit the incoming blocks.
- Each peer maintains a copy of ledger that includes chaincode as well.
- The ordering service receives incoming transactions from a client,orders them,creates a block and broadcasts it to the peers.
- A client application interacts with the blockchain network through APIs provided by Hyperledger fabric SDK.
- Peers may emit events to clients after committing a transaction.

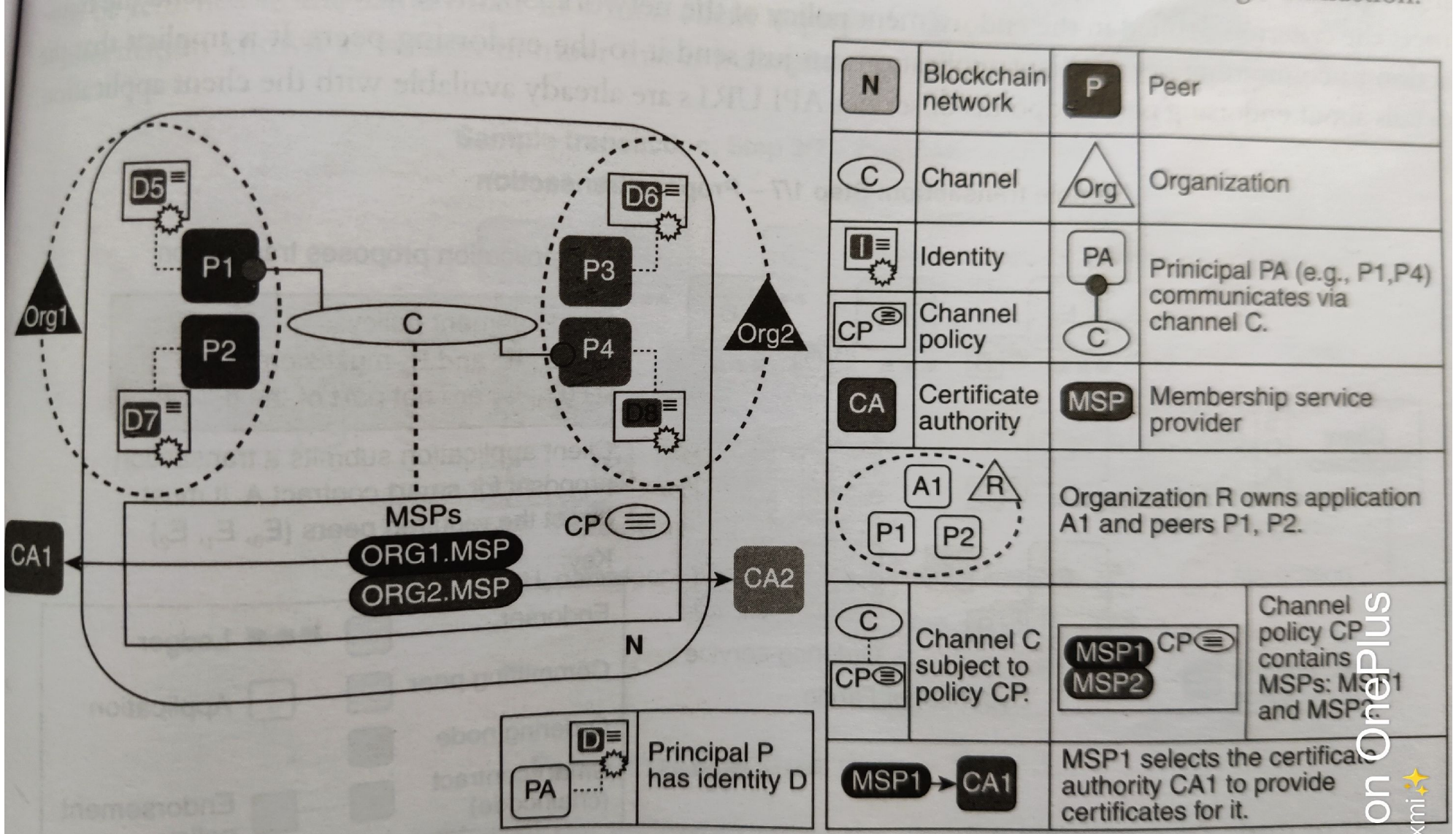
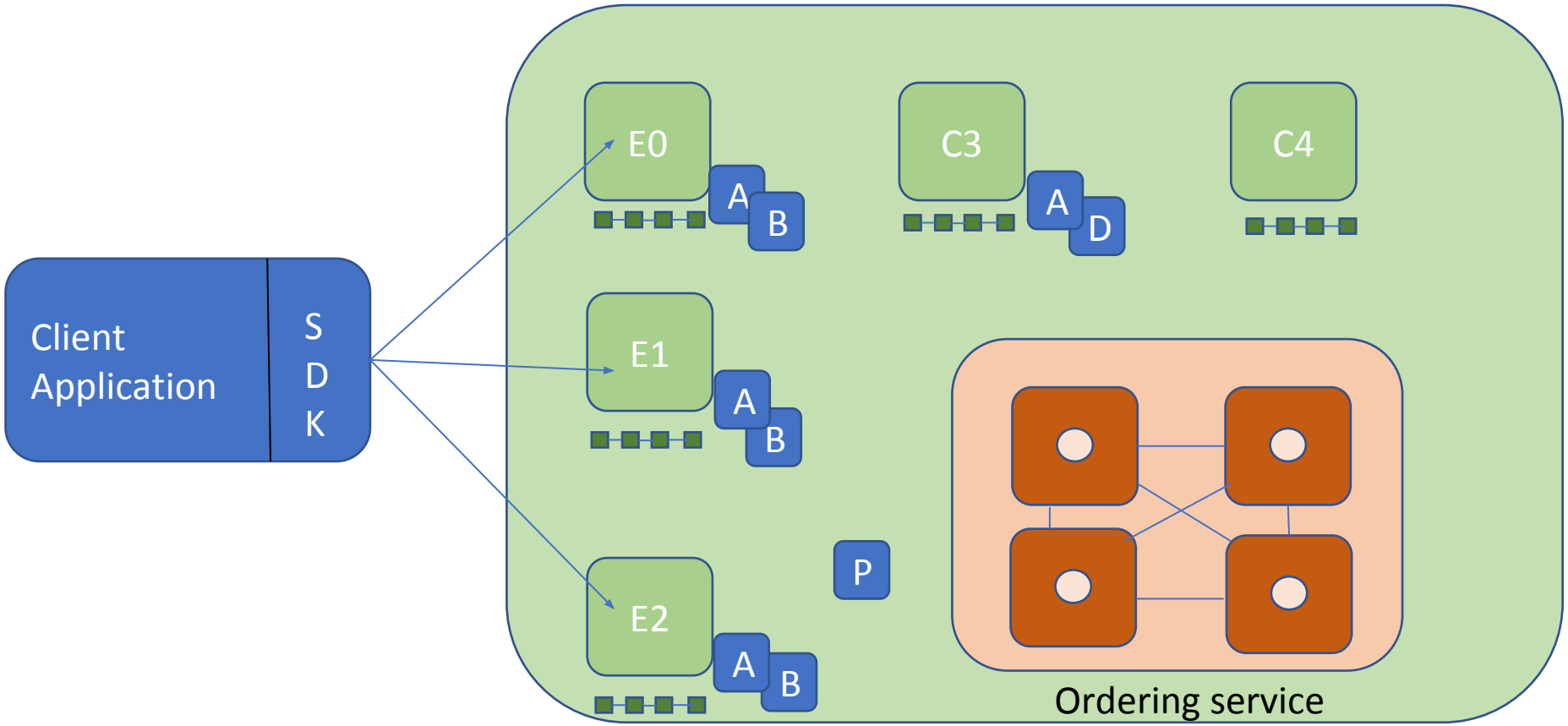


Figure 6.2 A Fabric network with two organizations.

Components of organization

Organization	Org 1	Org 2
Membership service providers(MSP)	MSP 1	MSP 2
Certificate Authority(CA)	CA 1	CA 2
Peers	P1,P2	P3,P4
Identities issued	P1->D5 P2->D7	P3->D6 P4->D8

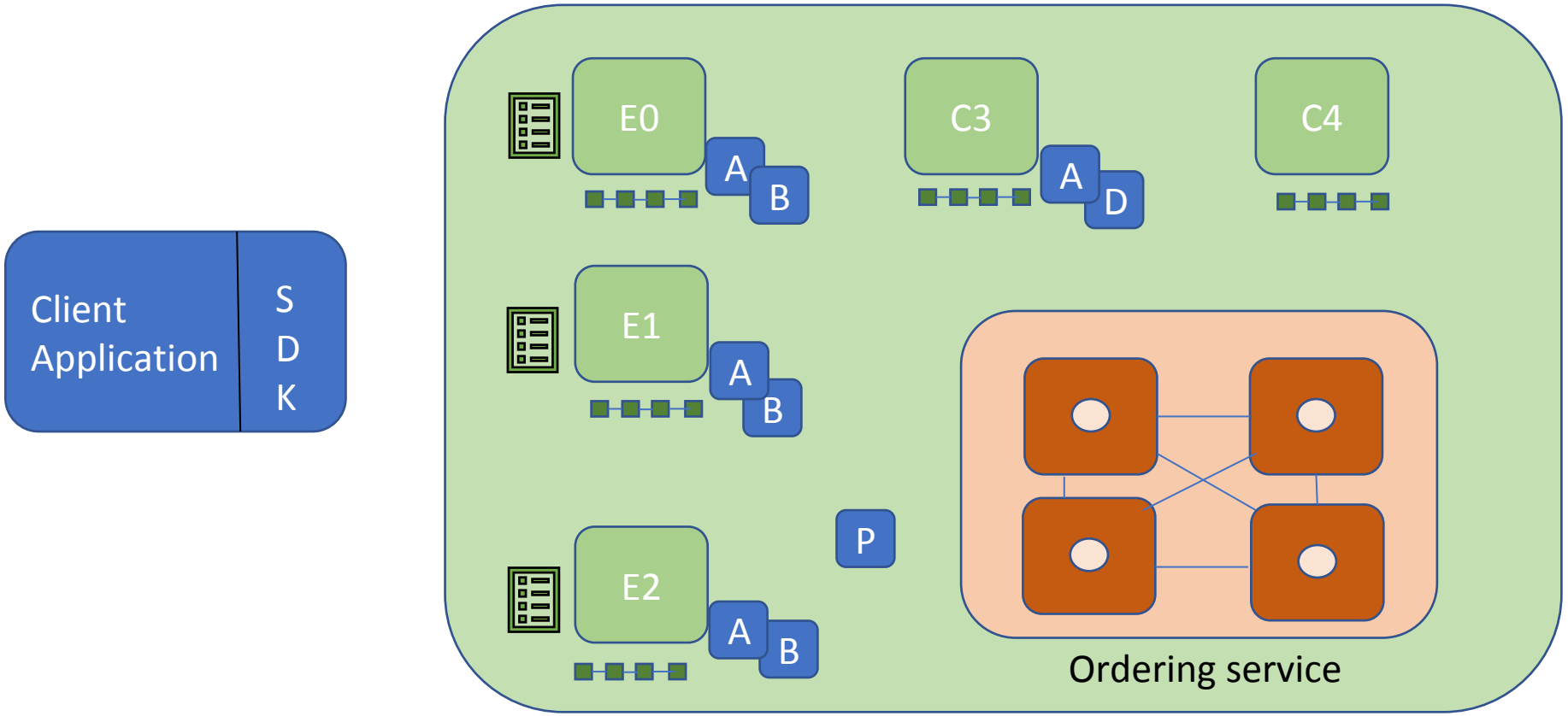
Hyperledger Fabric transaction flow –Step 1/7



Endorsement Policy:
"E0,E1,E2 must sign"
(C3,C4 are not part of the policy)

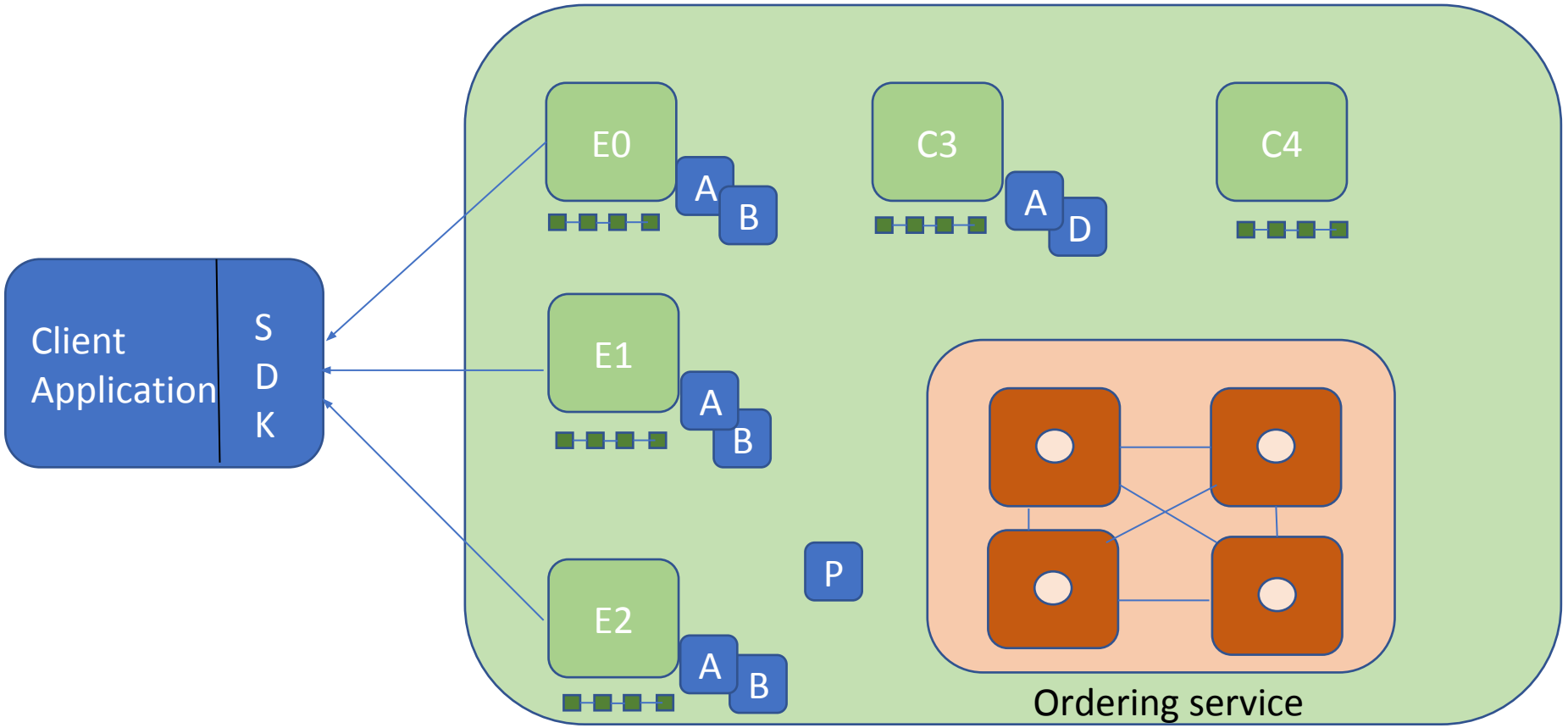
Client application submits a transaction proposal to smart contract A. It must target the required peers {E0,E1,E2}

Hyperledger Fabric transaction flow –Step 2/7



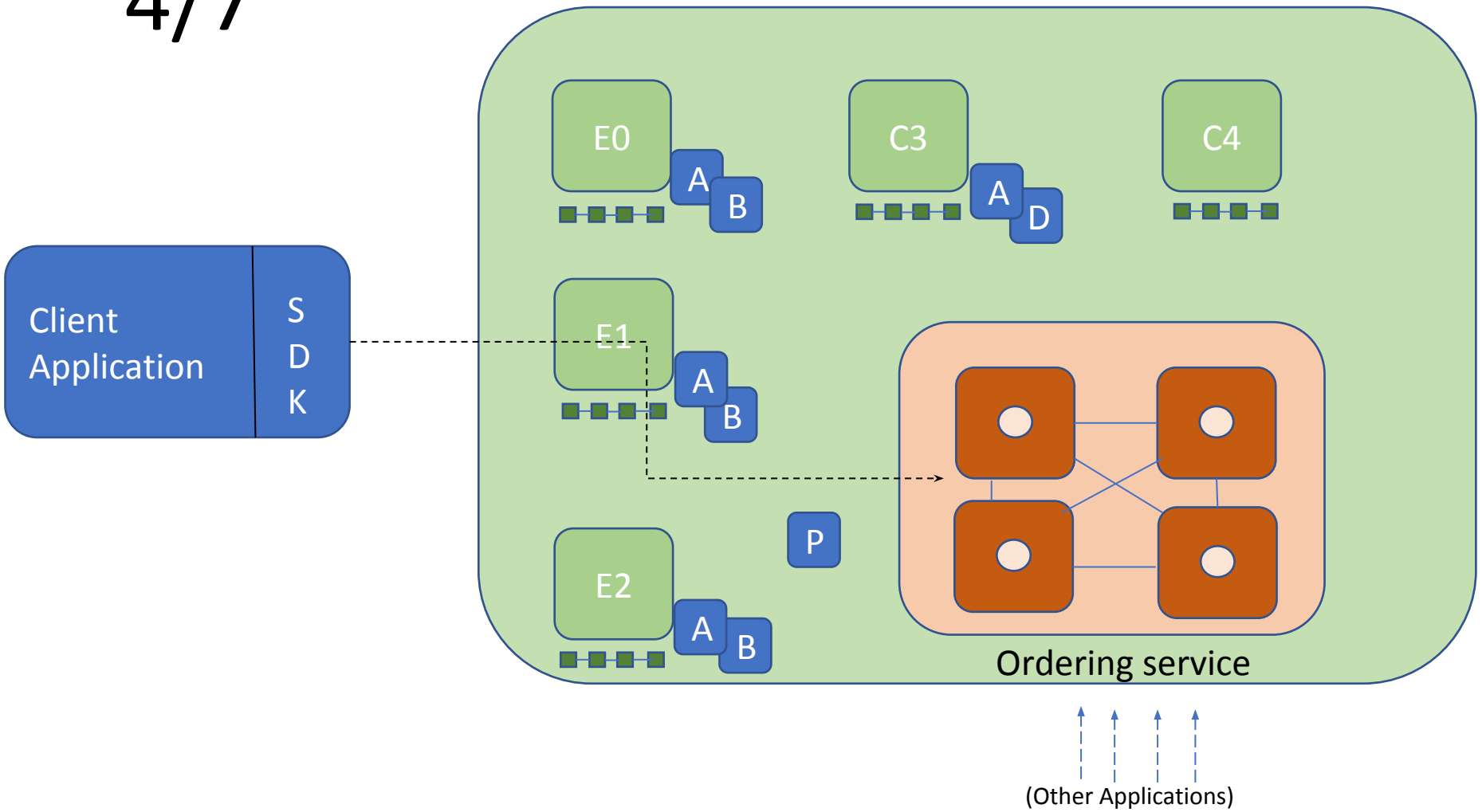
Endorser executes proposals E0,E1,E2 will each execute the proposed transaction. None of these executions will update the ledger.
Each execution will capture the set of read and written data, called RW sets, which will now flow in the fabric. Transactions can be signed and encrypted.

Hyperledger Fabric transaction flow –Step 3/7



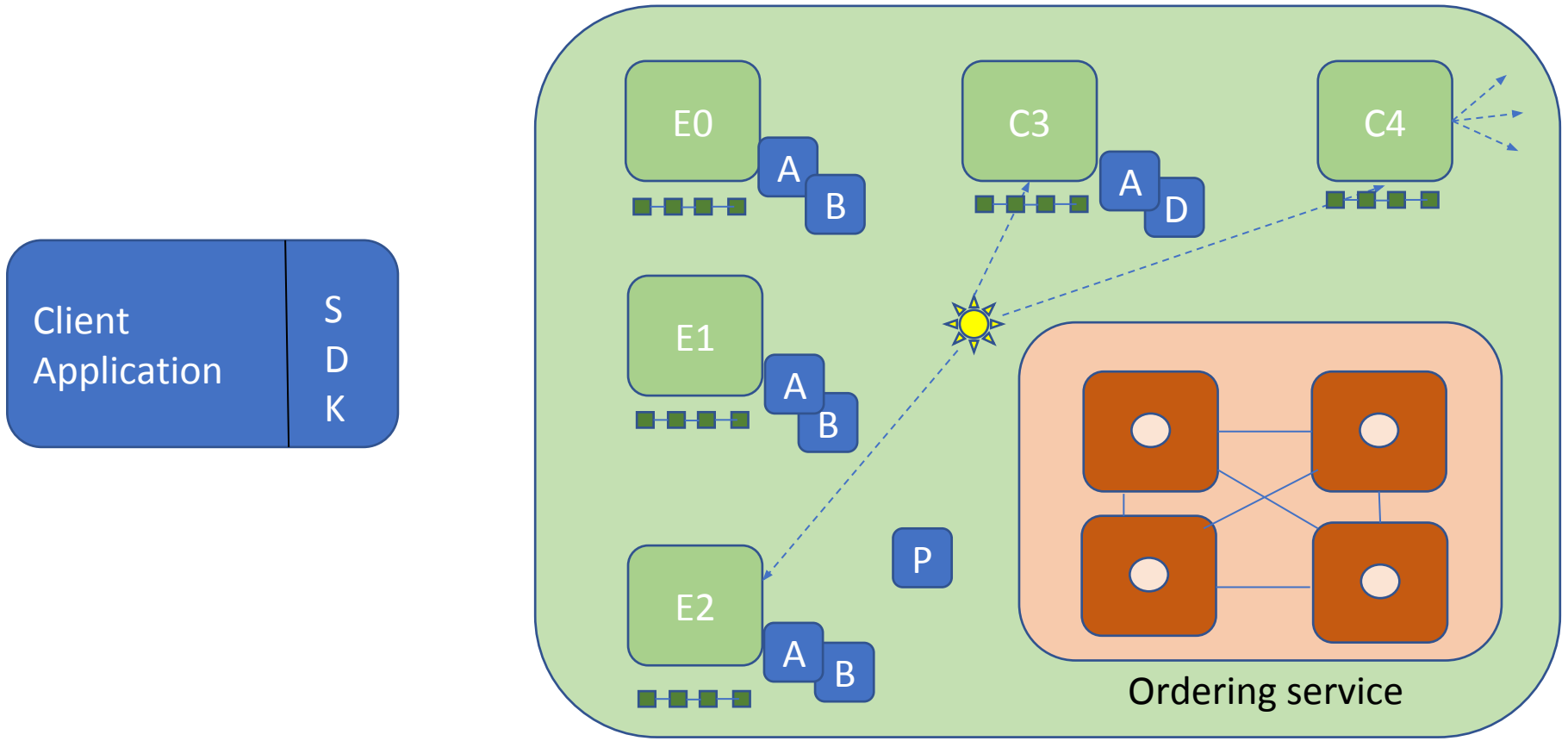
Application receives responses RW sets are asynchronously returned to application.
The RW sets are signed by each endorser,and also includes each record version number.
(This information will be check much later in the consensus process)

Hyperledger Fabric transaction flow –Step 4/7



Application submits responses for ordering
Application submits responses as a transaction to be ordered.
Ordering happens across the fabric in parallel with transactions submitted by other applications.

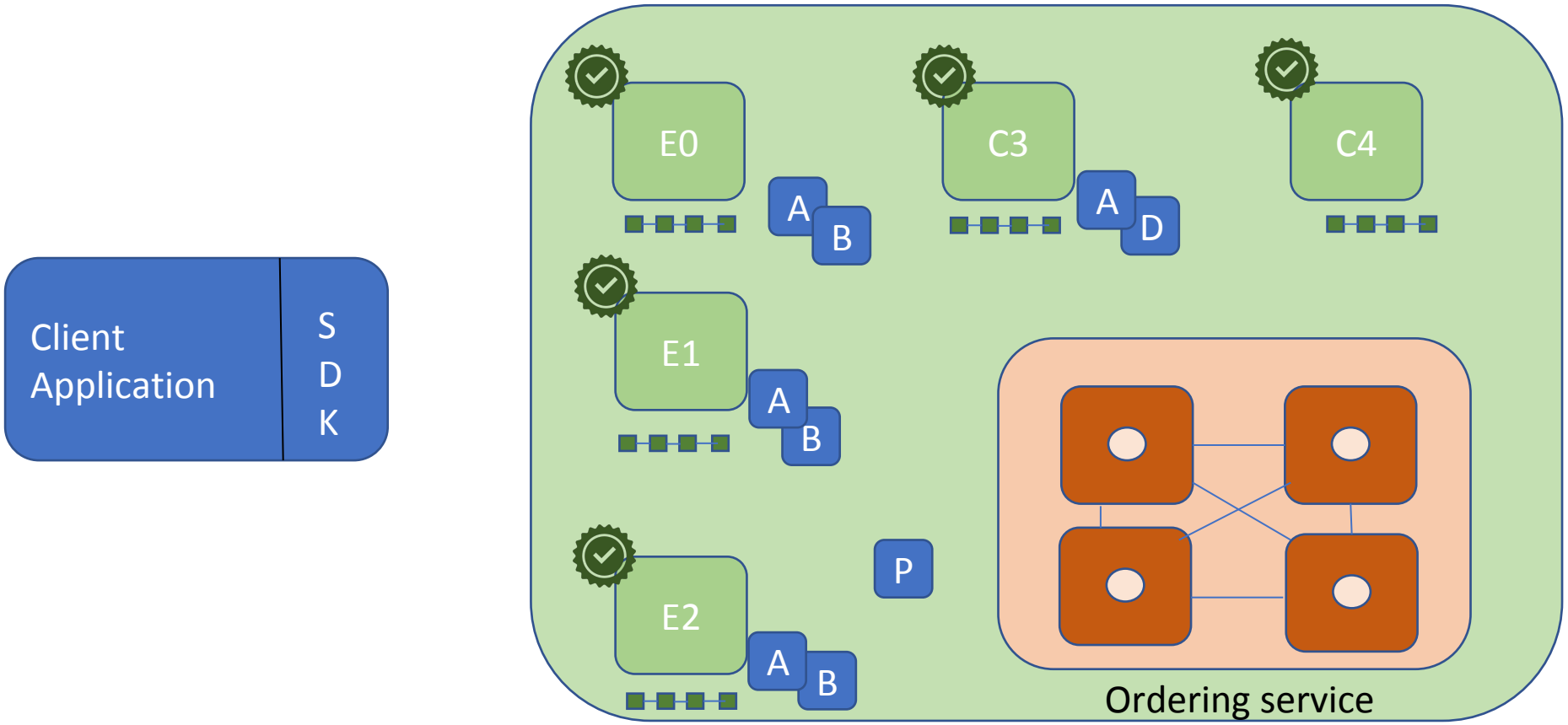
Hyperledger Fabric transaction flow –Step 5/7



Orderer delivers to all computing peers
Ordering service collects transactions into proposed blocks for distribution to committing peers.
Peers can deliver to other peers in a hierarchy(not shown).
Different ordering algorithms available:

- SOLO(Single node ,development)
- Kafka(Crash fault tolerance)

Hyperledger Fabric transaction flow –Step 6/7



Committing peers validate transactions

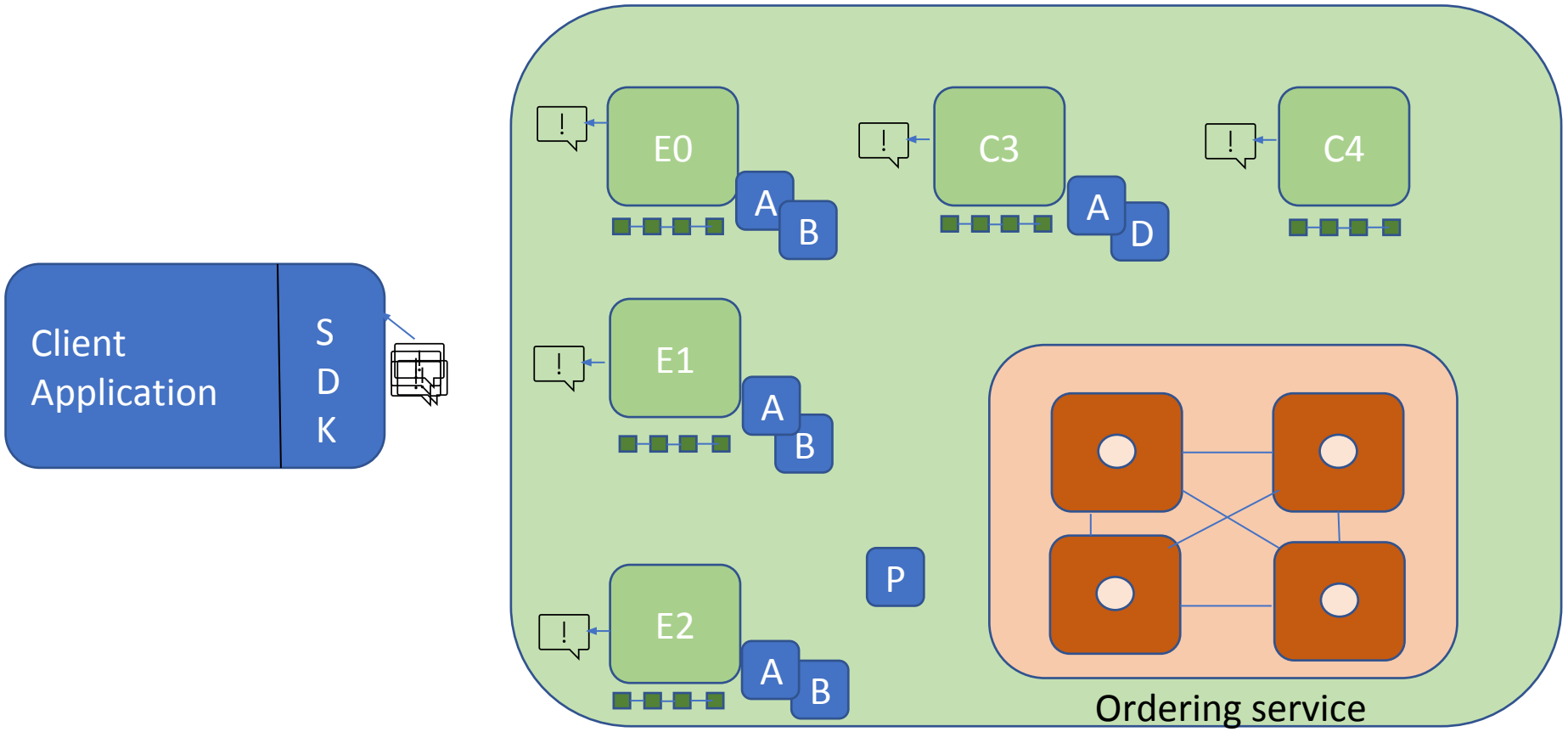
Every committing peer validates against the endorsement policy.

Also checks RW sets are still valid for current world state.

Validated transactions are applied to the world state and retained on the ledger.

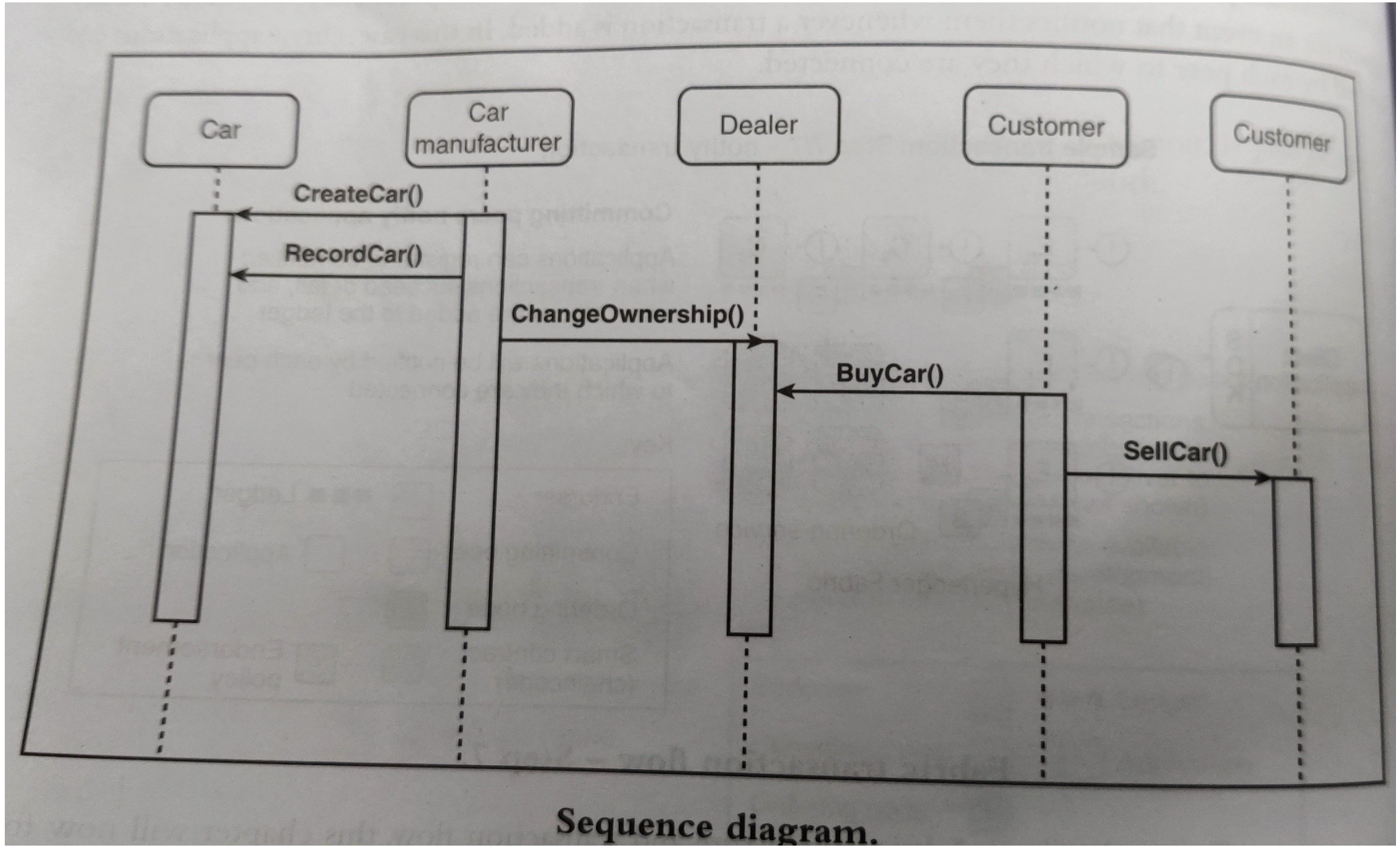
Invalid transactions are also retained on the ledger but do not update world state.

Hyperledger Fabric transaction flow –Step 7/7



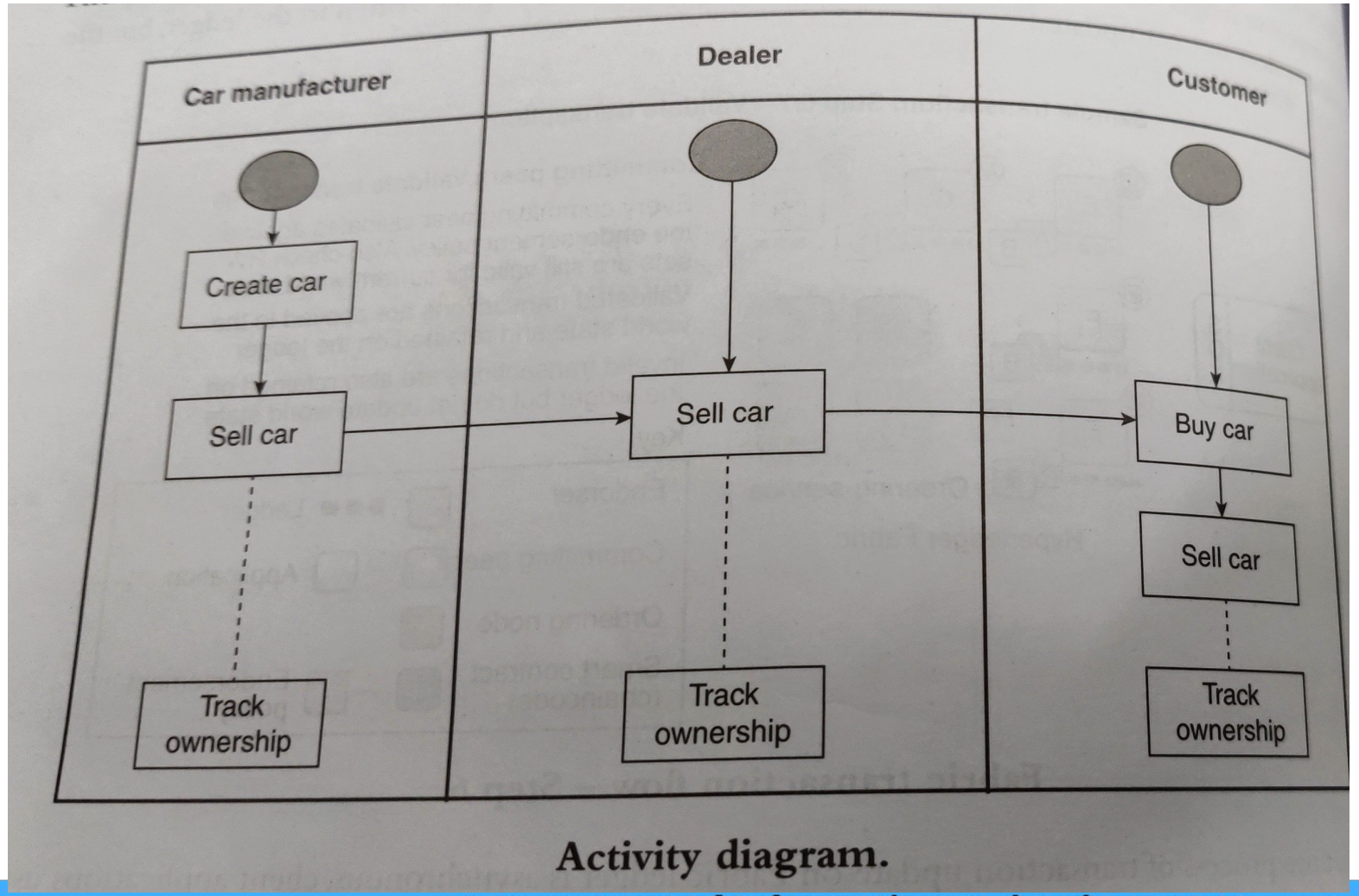
Committing peers notify applications
Application can register to be notified when transaction succeed or fail,and when blocks are added to the ledger.
Applications will be notified by each peer to which they are connected.

Fab Car sequence diagram



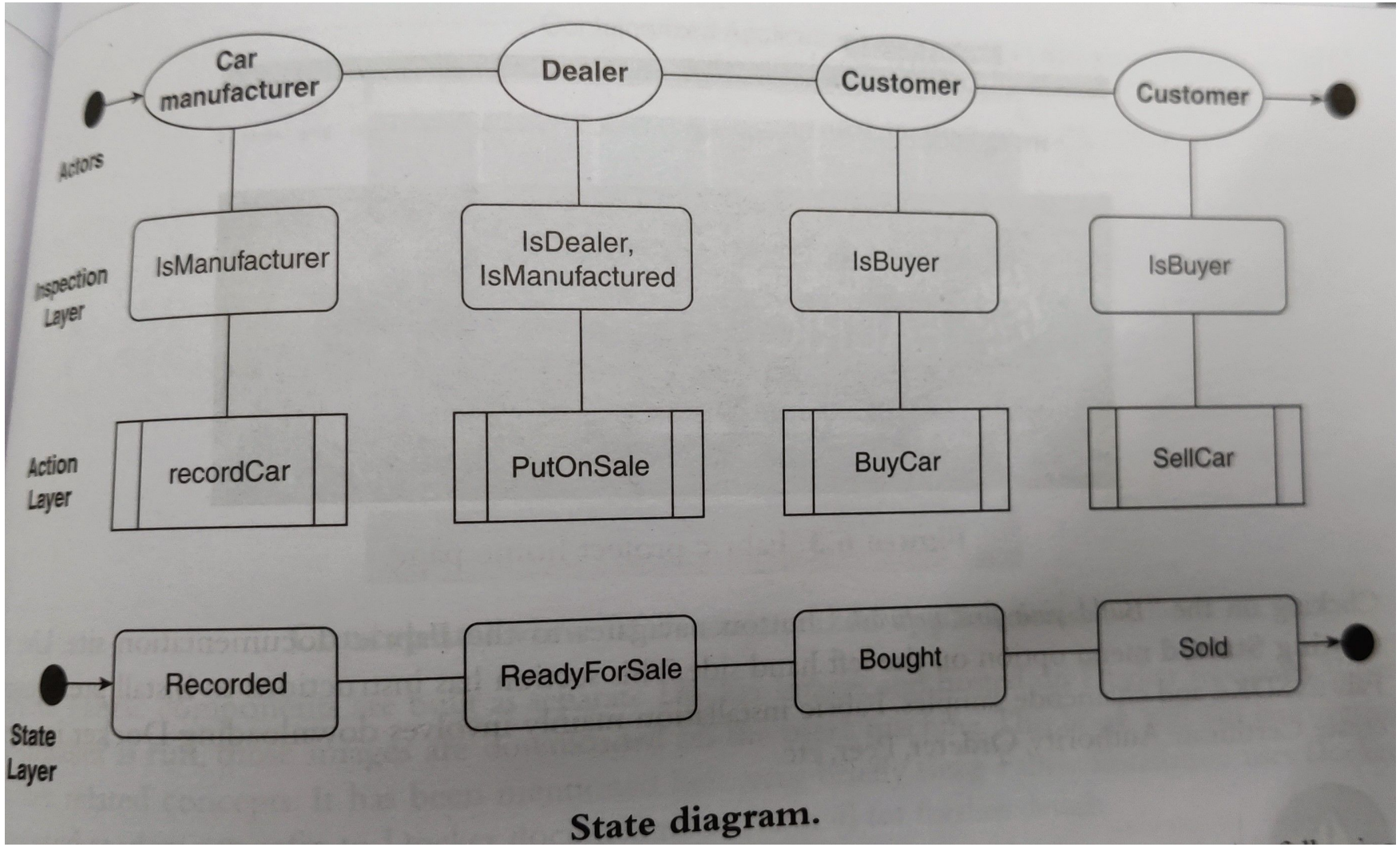
Sequence diagram.

Fab Car Activity diagram



Activity diagram.

Fab Car State diagram



State diagram.

Application development steps

- Installing Hyperledger fabric
- Setting the ledger
- Issuing identity to participants(peers and orderers)
- Setting the channel
- Defining access control list(ACL)
- Writing chaincode
- Invoking chaincode
- Creating APIs for client applications

(ACL is a set of rules used in Fabric to define and manage access to different resources/entities within a fabric network)

Installation

- Hyperledger fabric is an ongoing project from Linux foundation.
- The installation steps may subject to change in newer versions.
- So,it is recommended to visit Hyperledger website and follow steps.
  - https://www.hyperledger.org/
- Ensure sufficient disk space and good internet connectivity during download as Fabric image is of large size.
- Hyperledger fabric installation basically involves downloading Docker images and running them. Docker is a container-based architecture to package software units and resolve dependencies.

Chaincode programming

- Fabric supports Java,Node.js and Golang programming to write chaincode.
- Golang is a programming language developed by Google and also known as Go.
- It can be downloaded and installed as a standalone tool.
- However,it is included as part of Fabric installation.
- Hyperledger Composor,a tool from IBM can also be used to write chaincode.